

Comparative Performance Evaluation of Sparse, Dense and Hybrid Retrieval Models for Medical Search on NFCorpus

Student Name: _____

Registration No.: _____

Email-id: _____

Abstract — Medical search is hard for keyword matching because ordinary users and medical writers rarely use the same words. A person asks about a “heart attack” while the paper that answers them talks about “myocardial infarction”. This report builds and evaluates a search engine for the NFCorpus benchmark, which pairs plain-language health questions from NutritionFacts.org with 3,633 PubMed abstracts. Four ranking approaches are implemented and compared under one pipeline: a TF-IDF vector space model and Okapi BM25, both written from scratch on a hand-built inverted index; a dense Sentence Transformer retriever (all-MiniLM-L6-v2); and a hybrid that merges all three ranked lists with Reciprocal Rank Fusion (RRF). We also add medical query expansion based on pseudo-relevance feedback and on embedding neighbours. On the 323 test queries, BM25 reaches $nDCG@10 = 0.325$, which matches the published BEIR baseline, and the dense model reaches 0.319. Each of the two wins on exactly 108 queries, so their strengths barely overlap. RRF takes advantage of this: it lifts $nDCG@10$ to 0.352 (+8.1%) and MAP from 0.150 to 0.179. Adding pseudo-relevance feedback to the BM25 input of the fusion gives the best system overall ($nDCG@10 = 0.361$, MAP = 0.194, Recall@100 = 0.353). An interactive Streamlit interface lets the user run the models side by side.

Keywords: Information Retrieval · Medical Search · BM25 · Sentence Transformers · Reciprocal Rank Fusion · Query Expansion · NFCorpus

1. Introduction

Information retrieval (IR) is the task of finding, in a large collection, the documents that satisfy an information need expressed as a query. Search engines, question answering systems and Retrieval-Augmented Generation all depend on it. For decades the dominant approach has been *sparse lexical matching*: documents and queries are represented by the words they contain, an inverted index maps each word to the documents that use it, and a scoring function such as TF-IDF or BM25 rewards documents that contain the query words often and rare query words in particular. These models are fast, need no training and are easy to explain, which is why BM25 is still the standard baseline.

Their weakness is *vocabulary mismatch*. If the query and the relevant document describe the same idea with different words, a lexical model scores the document zero. Health search is a good example. Patients and the general public write “heart attack”, “gut bacteria” or “low-carb diets”, while the scientific literature says “myocardial infarction”, “intestinal microbiota” and “carbohydrate-restricted diets”. Dense retrievers based on transformer sentence encoders address this by mapping queries and documents into a shared vector space, where texts with similar meaning lie close together even if they share no words. Dense models have their own blind spot: rare technical terms, names and exact phrases are sometimes

blurred into general topics, so a query for a specific vegetable or chemical can return documents about related but wrong things.

Since the two families fail in different places, combining them is a natural idea. Reciprocal Rank Fusion (RRF) is a simple, training-free way to do this. It looks only at the *rank* of each document in each list, so it needs no calibration between BM25 scores (unbounded) and cosine similarities (between -1 and 1).

This project builds a complete medical search engine on NFCorpus and asks three questions: (i) how classical sparse models compare with a general-purpose dense model on lay medical queries, (ii) whether RRF actually gains from combining them, and (iii) whether medical query expansion helps further. Our contributions are:

- A from-scratch implementation of preprocessing, an inverted index, TF-IDF and BM25, together with a dense retriever and RRF, all evaluated with P@10, Recall@100, MAP and graded nDCG@10.
- Two medical query expansion methods: RM3-style pseudo-relevance feedback, and embedding-neighbour expansion that checks each candidate word against the whole query to avoid picking up the wrong sense.
- A per-query analysis showing *when* each model wins, and an interactive search interface that shows the models side by side.

2. About Dataset

NFCorpus (Boteva et al., 2016) is a medical IR benchmark. Its documents are PubMed abstracts cited by NutritionFacts.org, a website that explains nutrition research to the public. Its queries come from the same site: titles of videos and articles, topic pages, and plain questions such as “Do Cholesterol Statin Drugs Cause Breast Cancer?”. Relevance labels come from the citation structure. An article directly cited by the page that matches the query is graded **2** (highly relevant), and an article reached through one link is graded **1** (partly relevant). We use the version distributed in the BEIR benchmark (Thakur et al., 2021), loaded with the *ir_datasets* library, and evaluate on its official test split.

2.1 Visualization of dataset and an example

Table 1. NFCorpus test collection statistics.

Property	Value
Documents (PubMed titles + abstracts)	3,633
Test queries (with ≥ 1 relevant document)	323
Average document length	233.8 words (150.7 index tokens)
Average query length	3.3 words
Relevance judgments (grade > 0)	12,334
Average relevant documents per query	38.2
Grade 1 (partly relevant) / Grade 2 (highly relevant)	11,758 / 576

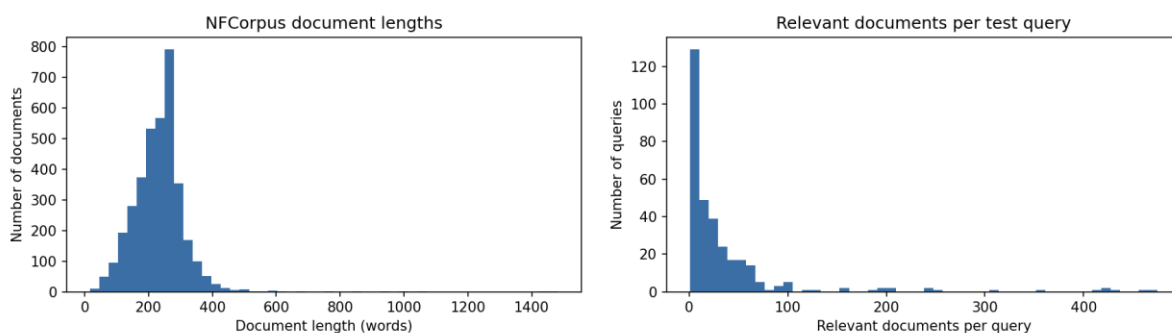


Fig. 1. Left: distribution of document lengths. Right: number of relevant documents per test query.

Three properties of the data shape the results. First, queries are very short: 172 of the 323 test queries have only one or two words (e.g. “kohlrabi”, “memory”, “milk”), so there is little text for either model to work with. Second, each query has many relevant documents (38 on average, and more than 100 for some broad topics), so Recall@100 is hard to push high and absolute scores are low compared with collections that have one answer per query. Third, 95% of judgments are grade 1, which is why we report graded nDCG@10: it rewards placing the few grade-2 documents first.

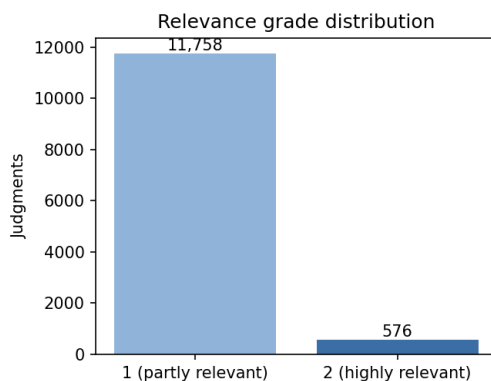


Fig. 2. Distribution of relevance grades in the test qrels.

Example. Test query PLAIN-2 is “Do Cholesterol Statin Drugs Cause Breast Cancer?” and has 24 relevant documents. One of them, MED-2427, begins: “Elevated Levels of Cholesterol-Rich Lipid Rafts in Cancer Cells Are Correlated with Apoptosis Sensitivity Induced by Cholesterol-Depleting Agents...”. After preprocessing (Section 3.1), the query becomes the token list [cholesterol, statin, drug, caus, breast, cancer], and the document title becomes [elev, level, cholesterol-rich, cholesterol, rich, lipid, raft, cancer, cell, ...]. The example shows why compound handling matters: without splitting “cholesterol-rich”, this document would not match the query term *cholesterol* in its title.

3. Methodology

All models share the same data loading, preprocessing and evaluation code, so differences in the results come only from the ranking function. Figure 3 shows the full pipeline. The sparse path (TF-IDF, BM25) runs on preprocessed tokens through the inverted index. The dense path gives the raw text to the transformer. RRF merges the three ranked lists. Query expansion is an optional step that rewrites the query sent to the sparse models.

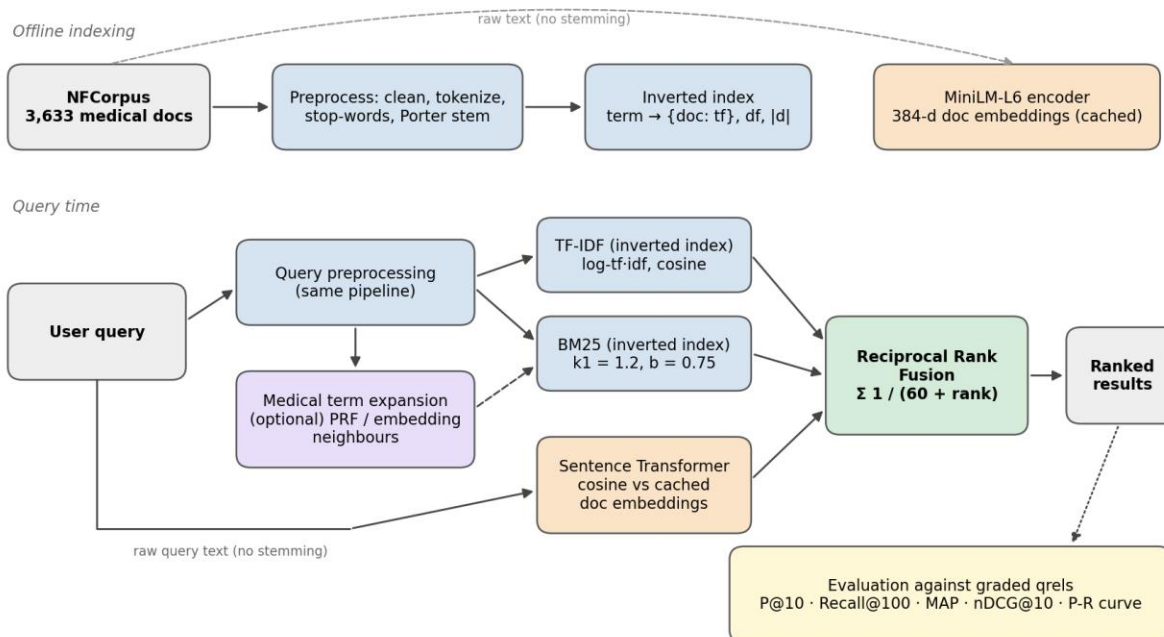


Fig. 3. Architecture of the hybrid retrieval pipeline, from indexing and query processing to rank fusion and evaluation.

3.1 Preprocessing and indexing

Documents (title followed by abstract) and queries go through the same steps: (1) lowercasing and removal of HTML remnants and punctuation; (2) tokenization with a regular expression that keeps alphanumeric and hyphenated tokens together, so that domain terms such as *omega-3*, *b12* and *covid-19* survive; (3) removal of the 198 NLTK English stop-words and of tokens that are only digits; and (4) Porter stemming, so that *drugs/drug* and *cause/causes/caused* map to one term. A hyphenated compound is indexed both whole and as its parts (“cholesterol-rich” → *cholesterol-rich*, *cholesterol*, *rich*), which keeps exact compound matches while still matching the parts.

The inverted index is built in a single pass over the corpus. For each term it stores a postings list of (document, term frequency) pairs, and it also keeps document frequencies, document lengths and the average document length needed by BM25. The final index has **22,913 terms** and **331,259 postings**. More than half of the terms (12,186) appear in only one document, which reflects how specialized the vocabulary is. At query time only the postings lists of the query terms are read, so sparse retrieval takes under half a millisecond per query.

3.2 Ranking models

TF-IDF vector space model. Each term in a document is weighted by logarithmic term frequency times inverse document frequency, and documents are ranked by cosine similarity with the query vector, which is weighted the same way:

$$w(t,d) = (1 + \log tf(t,d)) \cdot \log(N/df(t)), \quad score(q,d) = \cos(\vec{w}(q), \vec{w}(d)) \quad (1)$$

Okapi BM25. BM25 is a probabilistic model that adds two refinements to TF-IDF: term-frequency *saturation* (the tenth occurrence of a word adds much less than the first, controlled by k_1) and document-

length normalization (controlled by b). We use the common values $k_1 = 1.2$ and $b = 0.75$ and a smoothed IDF that never goes negative:

$$BM25(q,d) = \sum_{t \in q} IDF(t) \cdot tf(t,d) \cdot (k_1 + 1) / (tf(t,d) + k_1 \cdot (1 - b + b \cdot |d| / avgdl)) \quad (2)$$

$$IDF(t) = \ln(1 + (N - df(t) + 0.5) / (df(t) + 0.5)) \quad (3)$$

Sentence Transformer (dense retrieval). We use the pre-trained *all-MiniLM-L6-v2* bi-encoder (Reimers & Gurevych, 2019), which maps text to a 384-dimensional vector. It receives the original text, not the stemmed tokens, because the transformer was trained on natural language. All 3,633 documents are encoded once (about three minutes on a laptop CPU) and the L2-normalized vectors are cached. At query time the query is encoded and documents are ranked by cosine similarity, which for normalized vectors is a single matrix-vector product. The model reads at most 256 word pieces, so the tail of long abstracts is ignored, a limitation we return to in Section 5.

3.3 Reciprocal Rank Fusion formulation

RRF (Cormack et al., 2009) gives every document a score based only on its position in each input ranking. Given a set of rankers M and a smoothing constant k ($k = 60$ by default):

$$RRF(d) = \sum_{m \in M} 1 / (k + r_m(d)) \quad (4)$$

Here $r_m(d)$ is the 1-based rank of document d in the list of model m (documents missing from a list contribute nothing). We fuse the top 1,000 results of TF-IDF, BM25 and the dense model. Because only ranks are used, the scale mismatch between BM25 scores (values around 5–20 here) and cosine similarities (0.2–0.8) does not matter. The constant k flattens the curve, so that a document ranked 1st in one list and 50th in another can still beat a document that only one model likes. Section 4.4 measures how sensitive the results are to k .

3.4 Medical term expansion

The assignment brief asks for medical term expansion for this dataset. We implement two methods. Both produce a weighted query that is sent to TF-IDF and BM25. The dense model always sees the original query, since it already handles synonyms.

Pseudo-relevance feedback (PRF). Following the RM3 idea (Lavrenko & Croft, 2001), we run BM25, assume the top 10 documents are relevant, and score every term t in them as $\sum^d P(t|d) \cdot P(d|q) \cdot idf(t)$, where $P(t|d)$ is the term’s relative frequency in the document and $P(d|q)$ is the document’s normalized BM25 score. The idf factor stops generic words such as *study* and *result* from taking over. The 10 best new terms are added with weights up to 0.5 relative to the original terms, and BM25 is run again. For “obesity in children”, PRF adds *overweight*, *childhood*, *prevalence* and *adenovirus-36* (a virus studied as a cause of obesity in this corpus).

Embedding-neighbour expansion. We embed the 7,093 corpus words that occur in at least three documents. For each query word we take its nearest words in embedding space (cosine ≥ 0.6), but keep a candidate only if it is also close to the embedding of the *whole query* (cosine ≥ 0.35). Without this second check, “heart attack prevention” picked up *battle* and *threat* as neighbours of “attack”. With it, the expansion becomes *cardiac*, *myocardial*, *cardiovascular* and *precautions*, which is exactly the vocabulary-mismatch fix we want.

3.5 Search interface

The system is delivered as a Streamlit web application (Fig. 4). The user can pick a single model or a *Compare all 4* view, turn either expansion method on, and change the RRF constant k. Query terms are highlighted in yellow in titles and snippets, and expansion terms in blue. For any of the 323 test queries, each result is marked as relevant or not using the official judgments, and the per-query P@10, AP and nDCG@10 of every model are shown. Further tabs show the full evaluation tables and graphs, the dataset charts, and an inverted-index lookup.

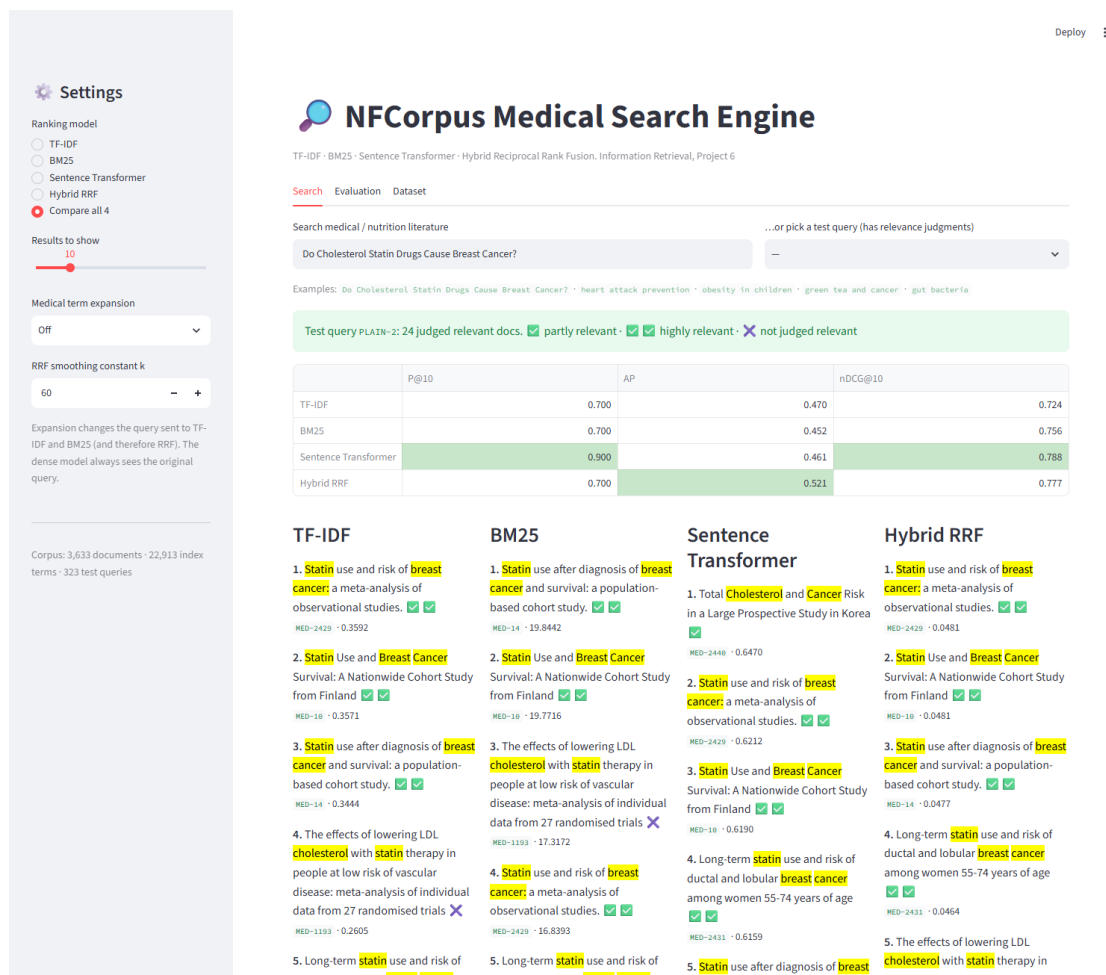


Fig. 4. “Compare all 4” view for test query PLAIN-2. Relevant documents are marked and per-query metrics are shown at the top.

4. Experimental Setup and Performance Analysis

Setup. Every model retrieves 1,000 documents for each of the 323 NFCorpus test queries. We report Precision@10 (fraction of the top 10 that are relevant), Recall@100 (fraction of all relevant documents found in the top 100), Mean Average Precision (MAP, over the full 1,000-document ranking) and nDCG@10 with graded gains $2^{\text{rel}} - 1$. All metrics are macro-averaged over queries. The 11-point interpolated Precision-Recall curve is averaged in the same way. Everything ran on a laptop CPU with no GPU or fine-tuning, and the whole evaluation finishes in under a minute once the embeddings are cached.

4.1 Main results

Table 2. Performance of the four core ranking models on the NFCorpus test set (best value in bold).

Ranking Model	Precision@10	Recall@100	MAP	nDCG@10
TF-IDF	0.2282	0.2470	0.1479	0.3167
BM25	0.2356	0.2476	0.1495	0.3253
Sentence Transformer (MiniLM)	0.2427	0.3149	0.1578	0.3190
Hybrid RRF (TF-IDF + BM25 + Dense)	0.2610	0.3100	0.1791	0.3517

Table 2 shows three things. First, the classical models are close to each other. BM25 beats TF-IDF on every metric, but only slightly, because NFCorpus documents have fairly uniform lengths (Fig. 1), so length normalization and saturation matter less than on web collections. Our BM25 score (nDCG@10 = 0.325) matches the BM25 figure published for NFCorpus in BEIR, which suggests the implementation is correct. Second, the general-purpose dense model is *not* better than BM25 on nDCG@10 (0.319 vs 0.325), even though it was trained on far more data. It does find much more of the relevant set (Recall@100 0.315 vs 0.248), which shows that it reaches documents that share no words with the query. Third, RRF is the best of the four on P@10, MAP and nDCG@10, improving nDCG@10 by 8.1% and MAP by 19.8% over BM25. The one exception is Recall@100, where RRF (0.310) is slightly below the dense model alone (0.315). Two of the three fused lists are lexical, so documents found only by the dense model get pushed below position 100 for some queries.

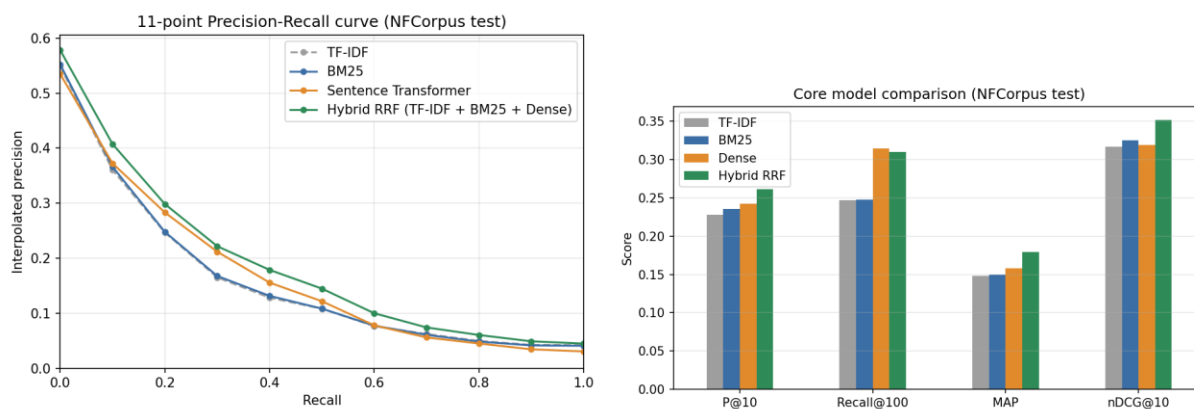


Fig. 5. Left: 11-point interpolated Precision-Recall curves. Right: metric comparison of the core models.

The Precision-Recall curves (Fig. 5) tell the same story at every recall level. TF-IDF and BM25 lie almost on top of each other. The dense model is clearly better in the middle of the curve (recall 0.2–0.6), where vocabulary mismatch starts to matter, and RRF lies above all three single models across the whole range.

4.2 Per-query analysis: why fusion works

Average scores hide a more interesting pattern. Comparing BM25 and the dense model query by query, each wins on **exactly 108** queries and they tie on the remaining 107 (mostly queries where both score zero). Table 3 shows how often each system beats another.

Table 3. Query-level wins / losses / ties on nDCG@10 (323 queries).

Comparison	Wins	Losses	Ties
Sentence Transformer vs BM25	108	108	107
Hybrid RRF vs BM25	119	54	150
Hybrid RRF vs Sentence Transformer	121	85	117
BM25 + PRF vs BM25	111	71	141
RRF (with PRF) vs RRF	82	65	176

The queries where the models disagree most show what each one is good at. BM25 wins on **rare, exact terms**: for “kohlrabi” BM25 reaches nDCG@10 = 1.0 while the dense model scores 0.0, and the same happens with “rickets” (0.47 vs 0.00) and “cadaverine” (0.64 vs 0.13). The dense model drifts to related topics (other vegetables, other vitamins). The dense model wins on **descriptive, title-like queries** that do not repeat the documents’ wording: “Vitamin D: Shedding some light on the new recommendations” scores 0.82 with dense retrieval but only 0.15 with BM25, which matches on words like *light* and *new*. RRF keeps most of the advantage in both directions: it keeps the perfect 1.0 for “kohlrabi” and reaches 0.70 on the vitamin D query. On 52 queries, RRF beats *both* of its inputs, for example “Does Cholesterol Size Matter?” (BM25 0.31, dense 0.43, RRF 0.62) and “memory” (0.58, 0.68, 0.85). In these cases each model finds a different part of the relevant set, and documents that both models rank fairly high rise to the top. RRF is worse than both inputs on only 9 queries. One case worth noting is “low-carb diets” (BM25 0.09, dense 0.66, RRF 0.27): here two weak lexical lists outvote one strong dense list. This is the main risk of giving every model an equal vote.

Table 4. nDCG@10 by query length.

Query length	Queries	TF-IDF	BM25	Dense	RRF	RRF + PRF
1–2 words	172	0.359	0.361	0.327	0.386	0.397
3–4 words	55	0.203	0.228	0.248	0.249	0.264
5+ words	96	0.306	0.317	0.345	0.350	0.351

Breaking results down by query length (Table 4) makes the trade-off clear. On one- and two-word queries, which are mostly specific foods, nutrients and conditions, the lexical models beat the dense model (0.361 vs 0.327). For queries of five or more words, which are mostly natural-language questions and article titles, the dense model is ahead (0.345 vs 0.317). RRF is the best single choice in every group, so a search engine that cannot know in advance which kind of query it will get is safer with fusion.

4.3 Effect of medical term expansion

Table 5. Query expansion and RRF variants.

Model	Precision@10	Recall@100	MAP	nDCG@10
BM25 (baseline)	0.2356	0.2476	0.1495	0.3253
BM25 + PRF expansion	0.2635	0.3268	0.1826	0.3468

Model	Precision@10	Recall@100	MAP	nDCG@10
BM25 + Embedding expansion	0.2409	0.2693	0.1587	0.3304
Hybrid RRF (BM25 + Dense only)	0.2576	0.3251	0.1842	0.3534
Hybrid RRF (TF-IDF + BM25+Emb + Dense)	0.2585	0.3167	0.1804	0.3513
Hybrid RRF (TF-IDF + BM25+PRF + Dense)	0.2666	0.3525	0.1941	0.3606

Pseudo-relevance feedback is the more effective of the two expansion methods. On its own it raises BM25 from 0.325 to 0.347 nDCG@10 and from 0.248 to 0.327 Recall@100, which closes the recall gap to the dense model. It helps on 111 queries and hurts on 71. The largest gains are on single-word topics where the top BM25 documents are on target and bring in their specialist vocabulary: “antinutrients” goes from 0.22 to 0.93, “apnea” from 0.47 to 0.97 and “aneurysm” from 0.36 to 0.82. The losses come from *query drift*. When the first-pass results are partly off-topic, the added terms pull the query further away, as with “shelf life” (1.00 → 0.63) and “Fish Fog” (0.25 → 0.00). Using PRF-expanded BM25 inside the fusion gives the best system in this study on all four metrics: **P@10 = 0.267**, **Recall@100 = 0.353**, **MAP = 0.194** and **nDCG@10 = 0.361**.

Embedding-neighbour expansion produces convincing synonyms (Section 3.4) but gives only a small gain for BM25 (0.325 → 0.330) and none inside RRF (0.3513 vs 0.3517). The likely reason is that its job overlaps with the dense model’s: once the dense retriever is in the fusion, adding “myocardial” to the BM25 query finds documents the dense model has already contributed. PRF, by contrast, adds terms taken from the corpus itself, including specific entities such as *adenovirus-36* that a general embedding model would not suggest. Its benefit therefore adds to the dense model’s rather than repeating it. Dropping TF-IDF from the fusion (RRF with BM25 + Dense only) slightly improves MAP and Recall@100 and leaves nDCG@10 essentially unchanged, which suggests that TF-IDF contributes little that BM25 does not already provide.

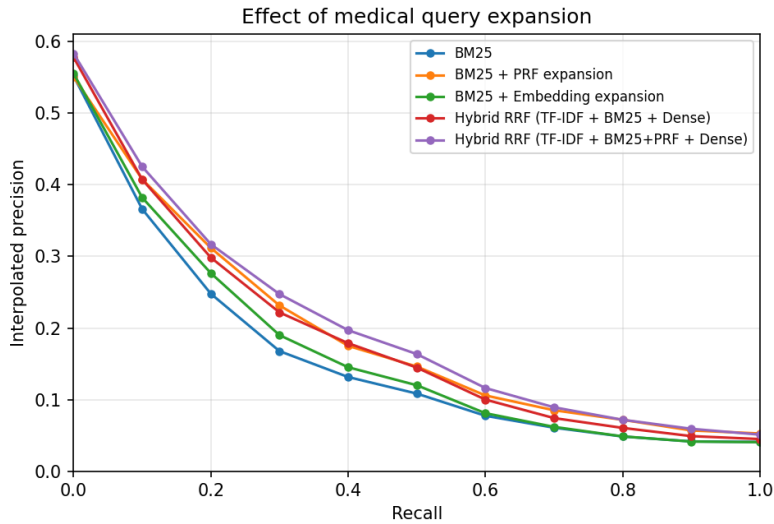


Fig. 6. Precision-Recall curves for the query expansion variants.

4.4 Sensitivity to the RRF constant and efficiency

Table 6. Effect of the RRF smoothing constant k (TF-IDF + BM25 + Dense).

k	Precision@10	Recall@100	MAP	nDCG@10
1	0.2557	0.3193	0.1787	0.3465
10	0.2563	0.3195	0.1805	0.3504
30	0.2579	0.3186	0.1798	0.3510
60	0.2610	0.3100	0.1791	0.3517
100	0.2598	0.3073	0.1785	0.3503
200	0.2591	0.3085	0.1779	0.3502

RRF is robust to its only parameter. Between $k = 10$ and $k = 200$, nDCG@10 stays within 0.350–0.352. Small k values give slightly higher Recall@100 because they reward the top of each list more strongly, and $k = 60$, the value recommended in the original paper, gives the best P@10 and nDCG@10. We therefore keep $k = 60$ without tuning it on the test set. In terms of speed, TF-IDF and BM25 answer a query in about 0.4 ms through the inverted index, the dense model needs about 11 ms (almost all of it encoding the query), PRF adds about 2 ms for its second retrieval pass, and the fusion itself costs almost nothing. The full hybrid system answers in well under 20 ms on a CPU.

5. Conclusion

We built a medical search engine for NCorpus that compares TF-IDF, BM25, a Sentence Transformer and Reciprocal Rank Fusion under one pipeline. The results support three conclusions. First, a general-purpose dense model is not automatically better than BM25 on specialist medical text: the two have almost identical average quality (nDCG@10 0.319 vs 0.325) but succeed on different queries, with BM25 winning on short, exact topic words and the dense model on longer, descriptive questions. Second, because their errors are largely independent, fusing them with RRF gives a clear and consistent improvement (+8.1% nDCG@10, +19.8% MAP over BM25) at almost no cost, and the result barely depends on the constant k . Third, medical term expansion with pseudo-relevance feedback adds to the hybrid’s gains and gives the best overall system (nDCG@10 = 0.361). Embedding-based expansion gives readable synonyms but little extra benefit once a dense retriever is already in the fusion.

The study has limitations. All results come from a single, fairly small collection; the dense model was used without any medical fine-tuning; and it sees only the first 256 word pieces of each abstract. RRF also gives every model the same vote, which, as the “low-carb diets” example showed, can let two weak lexical lists outvote one strong dense list. Natural next steps are: a biomedical encoder (e.g. PubMedBERT-based models) instead of MiniLM; a cross-encoder re-ranker applied to the fused top 100; weighted fusion, with weights chosen on the NCorpus development split; and expansion based on controlled vocabularies such as MeSH or UMLS, which would supply true medical synonyms rather than ones learned from text statistics.

References

1. Boteva, V., Gholipour, D., Sokolov, A., Riezler, S.: A Full-Text Learning to Rank Dataset for Medical Information Retrieval. In: Proceedings of the 38th European Conference on Information Retrieval (ECIR), pp. 716–722 (2016)
2. Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., Gurevych, I.: BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models. In: NeurIPS Datasets and Benchmarks Track (2021)
3. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3(4), 333–389 (2009)
4. Salton, G., Buckley, C.: Term-weighting Approaches in Automatic Text Retrieval. *Information Processing & Management* 24(5), 513–523 (1988)
5. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: Proceedings of EMNLP-IJCNLP, pp. 3982–3992 (2019)
6. Cormack, G.V., Clarke, C.L.A., Büttcher, S.: Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods. In: Proceedings of SIGIR, pp. 758–759 (2009)
7. Lavrenko, V., Croft, W.B.: Relevance-Based Language Models. In: Proceedings of SIGIR, pp. 120–127 (2001)
8. Järvelin, K., Kekäläinen, J.: Cumulated Gain-based Evaluation of IR Techniques. *ACM Transactions on Information Systems* 20(4), 422–446 (2002)
9. Porter, M.F.: An Algorithm for Suffix Stripping. *Program* 14(3), 130–137 (1980)